



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



Inteligencia Artificial

Práctica 2: Sistemas basados en el conocimiento



Jorge de la Fuente Martínez
Pol García Vernet
Roger Catalan Yarza

Índice

Identificación del problema y consideraciones del enunciado.....	2
Conceptualización de la ontología.....	5
Formalización.....	8
Fuentes de conocimiento.....	12
Implementación.....	14
Juego de pruebas.....	16
Conclusiones.....	18

Identificación del problema y consideraciones del enunciado

Una agencia de viajes planea construir un sistema de recomendación de planes a sus posibles clientes. En su catálogo tiene disponible múltiples ciudades y se tiene que indicar el medio de transporte utilizado para transportarse de una ciudad a otra, los días de estancia en cada ciudad, los alojamientos utilizados y los sitios a visitar de cada ciudad. Los clientes tienen una serie de restricciones como el presupuesto y un intervalo con la cantidad de ciudades que se desea visitar, además del tiempo de estancia en cada una.

Más allá de estos requisitos, pueden interesar otros factores para recomendar sitios más interesantes, como el propósito de su viaje (viaje de estudios, viaje de novios o viaje cultural, por mencionar algunos) para encontrar lugares que concuerden mejor con sus intereses. Además, pueden tener preferencias que se deberían cumplir si fuera posible: priorizar el tren sobre el avión porque alguna persona del grupo sufre mareos al volar, por ejemplo.

Usando el lenguaje de programación CLIPS, se debe construir un sistema que, a partir de una serie de preguntas iniciales, devuelva dos planes de viaje que cumplan las restricciones. En cada una de las sugerencias debe salir el precio por persona, las ciudades del circuito, el tiempo de estancia por cada ciudad, los sitios de alojamiento y los sitios de interés visitados. En caso de que no sea posible encontrar planes de viaje que satisfagan las restricciones, el sistema debe indicar su imposibilidad.

El enunciado de esta práctica es, evidentemente, una simplificación para poder realizar unos códigos y experimentos relacionados con sistemas basados en el conocimiento. El motivo de por qué lo es se comentará más adelante. Teniendo en cuenta dicha simplificación, se debe puntualizar una serie de consideraciones.

El conocimiento disponible en el sistema no tiene por qué ser información real. Según el propio enunciado, los datos pueden ser ficticios o incluso ser generados aleatoriamente. Dicha afirmación permite una cierta flexibilidad a la hora de realizar la práctica. Por ejemplo, como las distancias entre ciudades pueden ser ficticias, podría no considerarse el tiempo de traslado entre una ciudad y otra. Tampoco se especifica que se elabore un horario exhaustivo sobre qué hacer a cada hora; es más, en el ejemplo proporcionado se menciona sólo el número de días que se está en una ciudad, el alojamiento y los sitios que se visitan.

Los lugares pueden ser modificados en función de los intereses de los clientes. No es lo mismo una persona que prefiera visitar museos a otra que prefiere recorrer la ciudad a pie para contemplar su arquitectura. Así se podría ofrecer variedad para una misma ciudad.

En el enunciado no se considera importante de dónde parte el cliente, sino la primera ciudad del circuito turístico. En el ejemplo proporcionado queda reducido al nombre Origen. Así pues, como Origen es una abstracción que no hace referencia a ninguna particularidad de una ciudad real, se puede también simplificar considerando que se puede utilizar cualquier medio de transporte que también ofrezca cobertura en la primera ciudad del circuito, o en la última en el caso del viaje de vuelta. Dicho de otro modo, Origen es una ciudad abstracta que dispone de aeropuerto, red de carreteras, estación de autobuses y puerto.

Tampoco se ha tenido en cuenta la época del año en que se produce dicho viaje. Los precios de los alojamientos y de las visitas en varios sitios de interés varían en función de si es temporada alta o baja. También puede influir que se produzca un evento importante cerca de una ubicación: final de Champions, Gran Premio de Fórmula 1 o Festival de la Canción de Eurovisión, por citar algunos. También se considera que los hoteles y los medios de transporte registrados siempre van a tener plaza disponible para las personas que hacen la reserva.

Otro punto que puede resultar paradójico es la recomendación de lugares menos conocidos o relajantes. En lugar de ser medido un destino de forma dicotómica como *popular* o *poco conocido*, se utiliza como referencia la densidad de población. Ciudades con muchos habitantes como *Las Vegas* podrían entrar dentro del intervalo de destinos poco conocidos por tener una densidad de población más reducida. También puede ocurrir que una ciudad pueda tener muchos distritos, y que la densidad de población varíe. Zonas como la Rambla de Barcelona tienen una fuerte afluencia; y no sólo por su población, sino también por la masificación turística. A pesar del condicionante, este último factor no es tenido en cuenta.

Para no elevar la complicación del ejercicio, se ha considerado que todas las habitaciones de los hoteles poseen la misma capacidad. Además, no se tienen en cuenta otras consideraciones, como que una habitación para dos puede ser formada por una cama de matrimonio o por dos camas separadas. Los precios de todas las habitaciones de un hotel son iguales. También se podrían haber agregado otros sitios de alojamiento que no sean hoteles, como pueden ser pisos turísticos. Únicamente se han insertado hoteles por la facilidad a la hora de indicar la calidad del mismo, indicando el número de estrellas que

tiene. Otras características como la disponibilidad de internet, el tipo de menú o si aceptan niños no se han tenido en consideración.

En definitiva, para darle un punto de verosimilitud a la práctica, los datos insertados sobre las ciudades son reales. Sobre los hoteles, son reales el nombre y el número de estrellas que tiene. Sobre los sitios de interés turístico, son reales el nombre y el precio. El coste de los hoteles por una noche no es exacto, son los obtenidos para una habitación para dos personas en la fecha que se hizo la búsqueda en [booking.com](https://www.booking.com). Aunque en la mayoría de casos se considera que es el de cuatro personas. Todas las cantidades de la base de datos están expresadas en euros.

Se ha indicado que se va a utilizar un sistema basado en el conocimiento. Más allá de que el título de la práctica ya lo induce, ¿por qué se tiene que hacer así? Los sistemas basados en el conocimiento se basan en que existen dos entes: el experto y el ingeniero del conocimiento. El experto es conocedor del ámbito tratado, pero dicho conocimiento debe ser abstraído para poder obtener conclusiones a un problema de su ámbito de forma eficiente y comprensible. Esta es la labor del ingeniero del conocimiento, que trata de que este conocimiento pueda ser obtenido y simplificado a partir de una serie de reglas lógicas.

En este caso, el sistema basado en el conocimiento estaría basado en reglas de producción. El experto serían los datos sobre cada una de las ciudades, medios de transporte, alojamientos... En cambio, la ingeniería del conocimiento sería la organización de dichos datos mediante una ontología, que permita su representación, y la expresión de sentencias lógicas y condicionales para poder transformar dichos datos en un resultado que podría asemejarse al razonamiento de un experto, llamadas reglas de producción.

Construir un Sistema Basado en el Conocimiento es viable, más allá de su formalización, porque el conocimiento disponible es explícito y del ámbito tratado. A partir de la explicación anterior se entiende que se necesitan de dos entes: las reglas y los hechos, que almacenan de forma estructurada el conocimiento que se quiere gestionar. En el caso de esta práctica, la base de hechos inicial del sistema se almacena en un archivo llamado *conocimiento.clp*.

Conceptualización de la ontología

La ontología propuesta para la solución de este problema es el resultado de un proceso iterativo, en el que el primer paso ha sido identificar los términos clave, clasificando estos en clases o atributos y realizando una asociación entre las primeras.

A partir de la primera lectura, hemos identificado como clases las palabras clave Preferencia, Usuario, Restricción, Viaje, Objetivo y Ciudad. Esta primera iteración nos permite representar de una manera aproximada el escenario propuesto en el enunciado, ya que el usuario de nuestra herramienta quiere realizar un viaje a una o varias ciudades para alcanzar unos objetivos, dependiendo de un presupuesto y ajustándose a unas preferencias y su contraparte, restricciones.

Un usuario tiene unas preferencias que quiere cumplir durante el viaje. Estas preferencias pueden estar relacionadas tanto con algunos aspectos del viaje como los objetivos, y por lo tanto, afectan a qué ciudades podemos ofrecer en nuestras propuestas. En la misma línea lógica se aplican las restricciones, pues algunas de estas tienen impacto en las mismas clases. Sin embargo, vemos rápidamente que estas relaciones entre clases no se pueden representar directamente de forma precisa, ya que algunas de estas afectan a lo que ahora tenemos considerados como atributos. Por lo tanto, necesitamos realizar una segunda iteración sobre esta propuesta inicial.

Tras esta primera propuesta de ontología, y una vez empezamos a desarrollar las relaciones entre ellas, siempre basándonos en una mezcla entre el enunciado y la lógica, y descubrimos que, lo que inicialmente hemos clasificado como atributos, conviene ser transformado en subclases, ya que la relación entre clases es dependiente en algunos casos del valor de dichos atributos iniciales. Como consecuencia de esto, las siguientes clases se han desglosado en subclases:

- **Preferencia:** Se crean las subclases En Grupo, Solitario y Modo de Viaje.
 - El desglose “En Grupo” y “Solitario”, lo convertimos a “Tamaño de grupo”.
- **Ciudad:** Se crean las subclases Método de transporte, Sitios de Interés, Alojamiento
- **Restricción:** Se crean las subclases Duración, Número de paradas, Presupuesto y Tiempo en Ciudad.

Este desglose nos permite optimizar la conexión entre las nuevas clases, precisando mejor nuestra ontología al enunciado propuesto. Por ejemplo, el Tamaño de Grupo nos permite

obtener una relación directa entre esta clase y Alojamiento, o el Modo de Viaje puede afectar al Medio de Transporte utilizado para llegar a un nuevo destino Ciudad.

Habiendo realizado esta segunda iteración, decidimos que es momento de empezar a utilizar las herramientas actuales de modelos de lenguaje para expandir esta ontología y mejorar el conocimiento propuesto. Utilizando Gemini (como parte de las herramientas Google Suite del convenio con la universidad), identificamos nuevas consideraciones que debemos implementar en nuestra ontología que pueden ayudar a proponer un plan de viaje más realista: el tipo de presupuesto.

- Para un viaje, es importante tener en consideración qué elementos cubre un presupuesto, pues no es lo mismo un presupuesto que solo cubre desplazamientos que uno que incluya, además, estancia, avituallamiento y actividades.
- En el primer caso, el impacto sobre las propuestas de nuestra inteligencia puede implicar en habilitar opciones de Alojamiento con mayor coste, así como actividades no gratuitas o poder disfrutar de experiencias (ir de restaurantes, tomar copas en bares) que, en caso de tener que ser incluidas en el segundo caso, pueden implicar el no disfrutar de estas.
- Como consecuencia de esto, las subclases asociadas a una ciudad pasan a tener nuevos atributos derivados de estas consideraciones, tales como el coste de vida de la ciudad o el precio que puedan tener los sitios de interés y medios de transporte.
- Otros atributos añadidos como consecuencia de la interacción con el agente experto, son la densidad poblacional y el clima de la ciudad, que tienen impacto directo sobre el objetivo del viaje: una ciudad superpoblada, colapsada por el tráfico y calles abarrotadas de gente, no puede ser propuesto como un destino cuyo objetivo es el descanso.

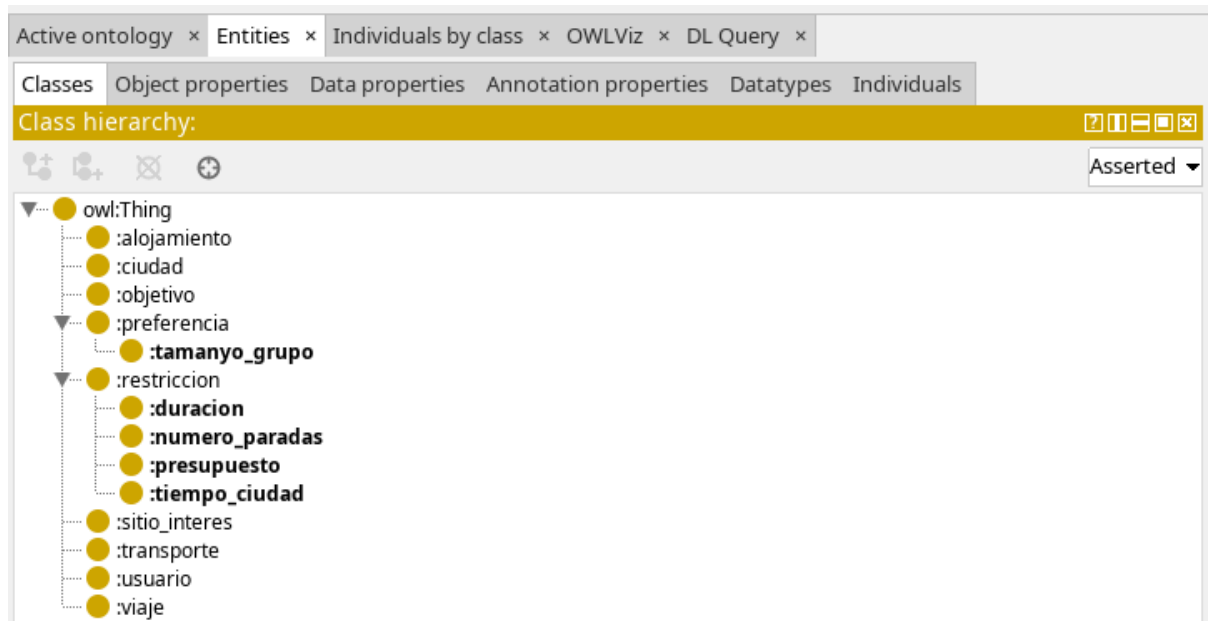
Sin embargo, para la entrega hemos realizado una cuarta y última iteración, que se ha realizado a partir de la formalización, donde hemos promocionado algunas subclases a clases y otras convertidas en atributos.



Formalización

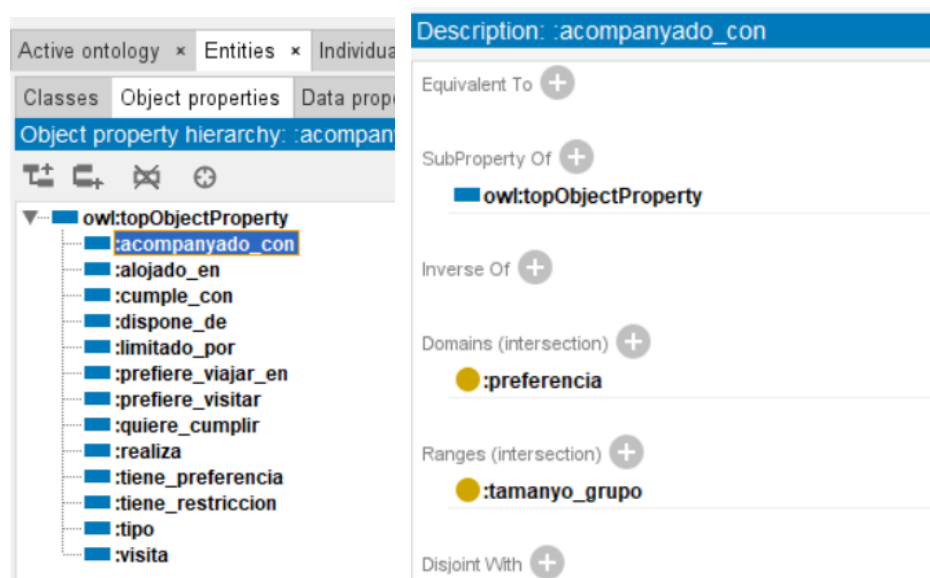
Una vez tenemos conceptualizada la ontología, es el momento de expresarla formalmente. Para esto, utilizamos el programa propuesto Protégé, que nos permite, de forma gráfica, aplicar el concepto y obtener la estructura sobre la que aplicar el conocimiento.

Lo primero que hacemos es crear las clases y subclases:

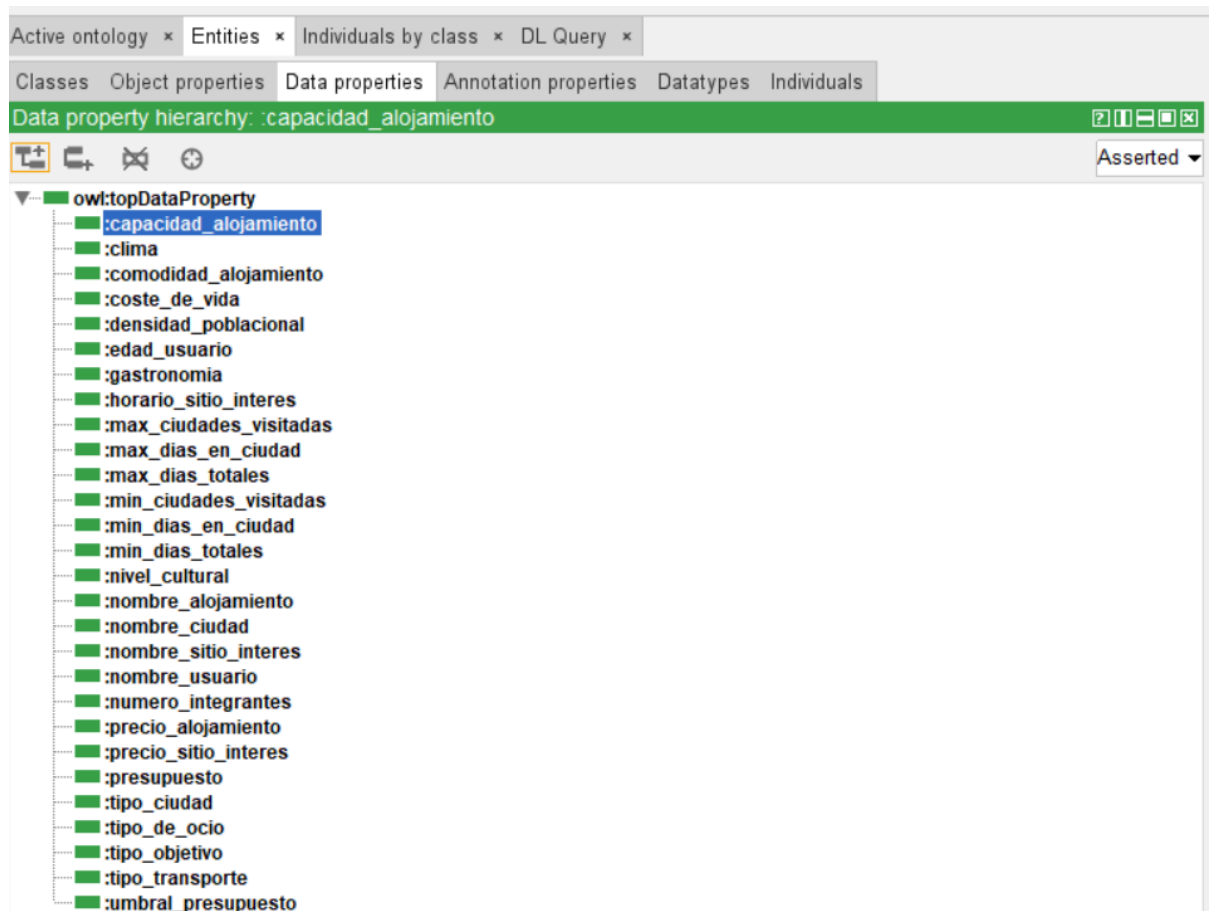


Este paso es bastante directo, pues ya disponemos del esbozo de la conceptualización, y no se requiere especificar más que el nombre.

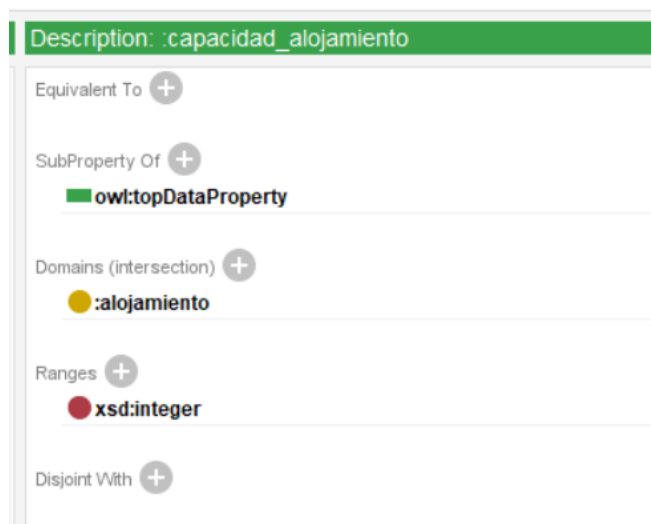
Una vez definidas las clases, debemos definir las relaciones entre clases. Nuevamente, al disponer del esquema de la ontología, es un proceso rápido. Una vez creada una relación, debemos definir qué clases conecta:



El tercer paso es definir los diferentes atributos, indicando a qué clase pertenecen y de qué tipo son. Primero, creamos los atributos:



Una vez creados, debemos asociarlos en su descripción para especificar la clase a la que describen, y qué tipo de datos utilizaremos para implementarlos:



A continuación, se muestra en una tabla las relaciones y atributos definidos en Protégé.

Para la tabla de Aristas:

Arista	Clase Origen	Clase(s) Destino
Acompañado con	Preferencia	Tamaño de grupo
Alojado en	Tamaño de grupo	Alojamiento
Cumple con	Sitio de Interés	Objetivo
Dispone de	Ciudad	Sitio de Interés, Alojamiento, Transporte
Limitado por	Viaje	Restricción
Viajar en	Preferencia	Transporte
Prefiere Visitar	Preferencia	Sitio de Interés
Quiere Cumplir	Viaje	Objetivo
Realiza	Usuario	Viaje
Tiene (Preferencia)	Usuario	Preferencia
Tiene (Restricción)	Usuario	Restricción
Visita	Viaje	Ciudad

Para la tabla de Atributos:

Atributo	Clase	Tipo
Capacidad de Alojamiento	Alojamiento	Integer
Clima	Ciudad	String
Comodidad de Alojamiento	Alojamiento	String
Coste de Vida	Ciudad	String
Densidad Poblacional	Ciudad	Float
Edad de Usuario	Usuario	Integer
Gastronomía	Ciudad	String
Horario del Sitio de Interés	Sitio de Interés	Date Time Stamp
Máximas Ciudades Visitadas	Número de Paradas	Integer
Máximo de Días en Ciudad	Tiempo en Ciudad	Integer
Máximo de Días Totales	Duración	Integer

Mínimo de Ciudades Visitadas	Número de Paradas	Integer
Mínimo de Días en Ciudad	Tiempo en Ciudad	Integer
Mínimo de Días Totales	Duración	Integer
Nivel Cultural	Usuario	String
Nombre del Alojamiento	Alojamiento	String
Nombre de la Ciudad	Ciudad	String
Nombre del Sitio de Interés	Sitio de Interés	String
Nombre del Usuario	Usuario	String
Número de Integrantes	Tamaño del grupo	Integer
Precio del Alojamiento	Alojamiento	Float
Precio del Sitio de Interés	Sitio de Interés	Float
Presupuesto	Presupuesto	Float
Tipo de Ciudad	Ciudad	String
Tipo de Ocio	Objetivo	String
Tipo de Objetivo	Objetivo	String
Tipo de Transporte	Transporte	String
Umbral del Presupuesto	Presupuesto	Float

Cuando toda la ontología está definida en Protégé, podemos obtener el archivo de ontología necesario para la fase de implementación.

Fuentes de conocimiento

El conocimiento disponible está en un archivo de texto llamado *conocimiento.clp*. Utilizando las clases implementadas en la ontología, se almacenan los siguientes datos:

- **Tipos de objetivo:** cada sitio de interés, por ejemplo, tiene que satisfacer ciertos aspectos. Por ejemplo, un museo tiene un propósito cultural o un parque puede ser relajante o romántico. En esta implementación existen tres instancias: Cultural, Descanso, Romántico [*sic*].
- **Tipos de Transporte:** las instancias Barco, Coche, Tren y Avion [*sic*].
- **Ciudades:** la base de hechos del conocimiento dispone de 21 ciudades. De cada una de estas se almacena lo siguiente:
 - **Nombre** de la ciudad.
 - **Clima.** En este caso, todas están escritas con uno de estos atributos: *mediterráneo, continental, atlántico, subtropical húmedo y árido*.
 - **Coste de vida.** Definido usando una de estas descripciones: *muy bajo, bajo, medio-bajo, medio, medio-alto, alto, muy alto*.
 - **Densidad poblacional.** Expresada en kilómetros por kilómetro cuadrado.
 - **Gastronomía.** descrita con un simple adjetivo: *japonesa, noreuropea, árabe...*
 - **Tipo de ciudad.** Una o dos palabras: *capital regional o cosmopolita*, por ejemplo.
 - **Elementos de los qué dispone** la ciudad:
 - Medios de transporte con los que se puede llegar hasta la ciudad.
 - Sitios de interés de la ciudad.
 - Alojamientos disponibles de la ciudad.
- **Sitios de interés:** están definidos por su nombre, el precio de su visita y el tipo de objetivo que se puede cumplir al explorar este sitio.
- **Alojamientos:** definidos a partir de su nombre, el precio por persona y noche y su calidad, medida en una nota de entre 1 y 5 estrellas.

La información de los alojamientos fue obtenida a partir de los precios en el momento que dichos datos fueron agregados a la base del conocimiento. La página de referencia es [booking.com](https://www.booking.com). Los precios de los sitios de interés fue obtenida de diversas fuentes, desde las oficiales de la institución como de páginas que escriben artículos de diversas ciudades turísticas. Una de estas es [viajeroscallejeros.com](https://www.viajeroscallejeros.com).

He aquí un ejemplo de instancia de ciudad, sitio de interés y alojamiento.

```
([Barcelona] of ciudad
  (nombre_ciudad "Barcelona")
  (clima "mediterráneo")
  (coste_de_vida "medio-alto")
  (densidad_poblacional 16600)
  (gastronomia "mediterránea")
  (tipo_ciudad "cosmopolita")

  (dispone_de
    [SagradaFamilia]
    ; ...
    [HotelWBarcelona]
    ; ...
    [Tren]
  )
)

([SagradaFamilia] of sitio_interes
  (nombre_sitio_interes "Sagrada Família")
  (precio_sitio_interes 50)
  (cumple_con [Cultural])
)

([HotelWBarcelona] of alojamiento
  (nombre_alojamiento "W Barcelona")
  (capacidad_alojamiento 1)
  (comodidad_alojamiento "5*")
  (precio_alojamiento 533.00)
)
```

Siendo Tren y Cultural instancias de Objetivo y Transporte, respectivamente.

Implementación

La lógica para recomendar viajes, dada la ontología *ontologia.clp* y los datos en *conocimiento.clp* está definida en *reglas.clp*. Cabe mencionar que si bien se ha seguido la abstracción, conceptualización y formalización de la ontología para la implementación, se ha permitido la libertad de realizar una serie de simplificaciones respecto a las clases Usuario, Presupuesto y Restricción. El Presupuesto y Restricciones que da el Usuario son variables que lee el programa, por lo que no hay instancias a diferencia de clases como Ciudad, Objetivo, Transporte o Alojamiento. Esta toma de decisión se debe a que resulta más sencillo leer el presupuesto, o cualquier otra restricción como la duración máxima del viaje, como variables durante la ejecución del programa que tener instancias dedicadas a ello. Otro detalle que debe ser mencionado es que debido a la falta de tiempo, se ha tenido que recortar algunas ideas representadas en la fase de conceptualización y formalización; como por ejemplo, el uso de umbral de presupuesto, horario del sitio de interés o el precio de este, o la edad del usuario.

La idea principal del algoritmo es realizar una serie de preguntas al usuario, como los días máximos que puede durar su viaje, alguna preferencia por el medio de transporte u objetivos de su viaje. Algunas preguntas y datos del cliente se deducen indirectamente. Por ejemplo, no se pregunta si el viaje es romántico si el tamaño del grupo no es 2 personas; o no se pregunta si se desean ciudades poco pobladas si el usuario especifica que su objetivo no es el de descanso. Una vez se procesan todas las preguntas, el algoritmo comienza con una lista con todas las ciudades de la base de datos.

La lista de ciudades se pasa por una serie de funciones que filtran aquellas ciudades que no cumplen los requisitos. Por ejemplo, si el usuario especifica que desea viajar únicamente en barco, se descartan ciudades no costeras. Según el nivel de exigencia del cliente, los filtros permitirán el paso de más o menos ciudades.

Una vez la lista se ha pasado por todos los filtros, comienzan las funciones de recomendación de viajes. Este es el núcleo del programa, que se encarga de seleccionar las ciudades, alojamientos y números de días que se destinarán en cada una de ellas en el viaje. Todos los algoritmos intentan maximizar los días de estancia en una ciudad; es decir, si el usuario desea estar entre 2 y 4 días en cada ciudad, se intentará recomendar planes con 4 días. Existen tres funciones de recomendación diferentes, todas ellas independientes y cuyo objetivo es el de ofrecer tres planes de viaje distintos:

1. `recomendar_viaje_ciudades`: se selecciona el número de ciudades que se intentará visitar, dentro del rango de ciudades mínimas y máximas que el usuario querrá visitar, de forma aleatoria. Se divide el presupuesto entre el número de ciudades para lograr un equilibrio económico entre todas las ciudades. Para decidir qué ciudades elegir, se itera en la lista filtrada y se busca el hotel más barato, respetando la comodidad mínima indicada por el usuario, y de forma que quepa en el presupuesto destinado en aquella ciudad. Si el coste excede el presupuesto parcial, se disminuye en 1 el número de días que se estará en la ciudad, y se repite la búsqueda. Si no es posible pagar bajo ninguna circunstancia, se ignora esa ciudad.
2. `recomendar_viaje_ciudades_inversa`: esta función es idéntica a la anterior, pero para añadir variedad en el plan de viaje, el índice en el que se explora la lista de ciudades es un índice invertido. Es decir, se inicializa como valor igual a la longitud de la lista y se decrementa su valor en cada iteración. El resto del algoritmo es idéntico y sigue los mismos criterios.
3. `recomendar_viaje_ciudades_premium`: esta función es idéntica a `recomendar_viaje_ciudades`, pero en vez de minimizar el coste, busca maximizarlo. Es decir, dada una ciudad, busca el alojamiento más caro. Cabe destacar que esta función también intenta maximizar los días de estancia en una ciudad. Uno de los inconvenientes de este método es que el gasto descontrolado en las primeras ciudades puede implicar que los viajeros se encuentren con un presupuesto ínfimo para los últimos días de vacaciones. Por ejemplo, si se recomienda Tokio y Barcelona, y el cliente exige un máximo de cinco días con un presupuesto de 3000 euros, la función buscará el hotel más caro de Tokio con una estancia de cinco días (si el presupuesto es suficiente) y consumirá parte del presupuesto. Podría darse el caso, por poner un ejemplo arbitrario, que la estancia de Tokio durante 4 días costara un total de 2800 euros, pero que paradójicamente se recomendará un hotel barato durante un solo día en Barcelona (para agotar los 200 euros restantes).

Finalmente, el resultado de las tres funciones es mostrado por pantalla junto a otras características de la ciudad, como la gastronomía, el tipo de ciudad, la densidad poblacional o sus sitios de interés y los objetivos que estos pueden cumplir, así como el lugar de alojamiento y el gasto total en aquella ciudad.

Juego de pruebas

Para probar el programa y su correcto funcionamiento, se han realizado una serie de pruebas con parámetros altamente diferentes. En el directorio test se puede encontrar diez juegos de pruebas que permiten ejecutar y observar el funcionamiento del código. Se prueban casos extremos con el objetivo de provocar que el programa falle o para que tenga un comportamiento no deseado. Los test están redactados en formato txt y se especifica qué respuestas se han de dar en cada pregunta, así como el output obtenido. Para probarlos, es necesario introducir manualmente los datos y comparar que la salida obtenida coincide con la indicada.

A continuación se explica el objetivo, entrada y salida esperada de cada uno de los juegos de pruebas:

- juego-prueba-0 : caso donde hay un único cliente con un altísimo presupuesto y con una exigencia de comodidad alta (cinco estrellas). Se espera una salida con recomendaciones con hoteles de alta calidad.
- juego-prueba-1: caso donde diez clientes tienen el presupuesto ínfimo de un euro. Se espera una salida con recomendaciones vacías; es decir, que no se encuentra ningún plan de viaje.
- juego-prueba-2: caso donde dos clientes tienen muchísimas exigencias. Solo desean viajar en barco, quieren objetivos culturales, románticos y de descanso, baja densidad poblacional, sólo desean visitar ciudades con un clima en particular, y exigen una comodidad de cinco estrellas. Se esperan recomendaciones con una cantidad baja de destinos.
- juego-prueba-3: caso donde dos clientes dan información posiblemente conflictiva. Se exige viajar únicamente en barco, pero al mismo tiempo se restringe el clima a que solo sea continental. Se espera que no se obtengan recomendaciones.
- juego-prueba-4: caso donde dos clientes dan información conflictiva. Exigen viajar por más ciudad que el máximo de días indicado. Se esperan recomendaciones que incumplan la restricción de la cantidad de ciudades.
- juego-prueba-5: caso con información errónea. Los mínimos son mayores que los máximos. Se espera que el programa avise de los datos erróneamente introducidos.

- juego-prueba-6: caso donde el cliente da información posiblemente conflictiva. Se indica que no se quiere viajar en coche, en avión, en tren ni en barco. Se espera que no se encuentren recomendaciones.
- juego-prueba-7: juego de prueba de un caso realista. Una familia quiere viajar con un presupuesto ajustado para cinco personas. Se esperan recomendaciones que cumplan con los requisitos.
- juego-prueba-8: juego de prueba de un caso realista. Una pareja quiere viajar con un presupuesto alto y desea lugares poco transitados. Se esperan recomendaciones que cumplan con los requisitos..
- juego-prueba-9: juego de prueba de un caso realista. Una excursión de un gran grupo, quince personas, con un presupuesto de 100 euros por persona. Se esperan recomendaciones que cumplan con los requisitos.

Conclusiones

La elaboración de esta práctica no sólo ha supuesto el reto como puede ser trabajar con un nuevo lenguaje, sino que supone un cambio de paradigma. Tanto la industria como los proyectos individuales (ya sean de código abierto o no) están dominados por los lenguajes de paradigma imperativo. En este sentido, CLIPS es declarativo y derivado de LISP, uno de los primeros lenguajes que se crearon, caracterizado por la presencia de paréntesis en la apertura y cierre de una expresión, que sigue una notación prefija. Aunque no sea estrictamente declarativo, ya que tiene características del paradigma imperativo (como la declaración de variables) es un cambio de mentalidad diferente. Esa ha sido una de las mayores dificultades en el avance del proyecto.

A pesar de la indeseada adaptación a CLIPS, por otro lado, se ve la capacidad expresiva del mismo. En un lenguaje imperativo convencional se necesita marcar, instrucción por instrucción, el control de flujo del programa y las variables. En este caso, la programación es de bastante más alto nivel. Ayudado por características que heredan de otros lenguajes, como la programación orientada a objetos, ha dado lugar a una mayor simplificación en la escritura de código. Algo que no resulta posible usando un lenguaje como Python, C++ o Java.

Finalmente, se ha de reconocer que no ha sido un proyecto sencillo. La falta de tiempo ha condicionado a recortar bastantes ideas pensadas en la conceptualización y en la formalización. Una de estas propuestas es tener en cuenta los horarios de apertura de los sitios de interés, la posibilidad de un umbral presupuestario (por si se encuentra un buen plan de viaje que cueste unos euros más que el presupuesto inicial) que determine cierta flexibilidad por parte del usuario. Tampoco se tiene en cuenta la edad de los usuarios que viajan y la modificación del precio de visita en función de la edad.